

Relatório de evolução — Weely / ChargeGrid Intelligence

EV Challenge — GoodWe · Sprint 03 · Agentes de IA e Evolução Conversacional

7.1 Resumo da evolução

Sprints 1 e 2: o grupo definiu o problema (gestão e monetização de infraestrutura de carregamento veicular para estabelecimentos comerciais), a persona (dono/gestor do estabelecimento), o system prompt e as perguntas de avaliação; em seguida implementou um chatbot funcional em notebook (Google Colab), chamando diretamente a API `chat.completions` da OpenAI (`gpt-4o-mini`) e gerenciando o histórico manualmente em uma lista de mensagens (`msgs.append(...)`).

Sprint 03: o núcleo conversacional foi refatorado para uma **aplicação local em Python** construída sobre o **OpenAI Agents SDK**. O que foi adicionado ou modificado:

- **Framework de agentes** orquestrando a conversa (`Agent + Runner`);
- **Memória por sessão** gerenciada pelo framework (`SQLiteSession`), persistida em SQLite e isolada por identificador de sessão;
- **Guardrail de entrada** (agente classificador que bloqueia prompt injection e mensagens fora de escopo antes de chegarem ao modelo principal) + regras de segurança reforçadas no system prompt;
- **Suíte de testes automatizada** (funcionais, memória e segurança) com avaliação por LLM-as-judge, medição de latência e tokens;
- **Experimentação com múltiplos modelos/parâmetros** e escolha baseada em métricas (`relatorio_modelos.md`);
- **Gestão segura de credenciais** via `.env` (fora do Git).

7.2 Refatoração — decisões técnicas e trade-offs

Framework escolhido: OpenAI Agents SDK. Motivos:

1. A Sprint 2 já usava a API da OpenAI — a migração aproveita a mesma conta, chave e modelos, permitindo comparação justa antes × depois;
2. Os mecanismos exigidos pela sprint são nativos do framework: sessões de memória (`SQLiteSession`), guardrails de entrada (`@input_guardrail` com `tripwire`), configuração declarativa de modelo/parâmetros (`ModelSettings`) e medição de uso (`usage`);
3. API enxuta — o grupo consegue explicar cada componente, o que não seria trivial com a pilha maior do LangChain/LangGraph.

Componentes utilizados: `Agent` (Weely, guardrail classificador e dois juízes de avaliação), `Runner`, `SQLiteSession`, `@input_guardrail` / `GuardrailFunctionOutput` / `InputGuardrailTripwireTriggered`, `ModelSettings` e `output_type` (saídas estruturadas com Pydantic para os juízes e o classificador).

Principais trade-offs:

- *Acoplamento ao ecossistema OpenAI:* o SDK é da OpenAI; outros provedores exigem a extensão LiteLLM. Aceitamos porque o projeto já dependia da OpenAI.
- *Custo/latência do guardrail:* cada turno dispara uma chamada extra ao classificador (`gpt-4o-mini`). Aceitamos: o classificador é barato, bloqueia ataques **antes** de gastar tokens do modelo principal e centraliza a segurança fora do system prompt.

- *Falsos positivos do guardrail*: um classificador agressivo poderia bloquear perguntas legítimas; mitigamos instruindo-o a marcar "na dúvida, false" e validamos com os testes funcionais.
- *LLM-as-judge*: notas automáticas podem oscilar; mitigamos com critérios objetivos por caso e revisão manual das justificativas gravadas nos JSONs.

7.3 Comparativo antes x depois

Para o "antes" ser mensurável, uma **réplica da Sprint 2** (system prompt original do notebook, sem guardrails e sem regras de segurança, gpt-4o-mini t=0.7) foi executada contra a mesma suíte de testes (tests/resultados/baseline_sprint2.json).

Dimensão	Sprints 1–2	Sprint 03
Arquitetura	Notebook Colab, loop com <code>chat.completions</code> manual	Aplicação local modular sobre OpenAI Agents SDK
Memória	Lista <code>msgs</code> em RAM, perdida ao fechar o notebook	<code>SQLiteSession</code> por sessão, persistente em disco (10/10 nos testes M1–M2)
Segurança	Apenas instruções no system prompt	Guardrail de entrada + regras explícitas + 7 testes de segurança
Credenciais	<code>userdata</code> do Colab	<code>.env</code> + <code>.gitignore</code>
Modelo	<code>gpt-4o-mini t=0.7</code> (escolha sem experimentação)	<code>gpt-4o-mini t=0.2</code> (escolhido após comparar 3 configurações com métricas)
Avaliação	Leitura manual das respostas	Suíte automatizada + juiz LLM (nota 0–10)
Nota funcional média	9.6/10	9.2/10 (equivalente; diferença causada por 1 caso borderline de persona, F5)
Testes de segurança adequados	4/7 (inventou especificações do HCA-G2; deu orientação jurídica/tributária; sugeriu análise financeira sem ressalvas)	7/7 (prompt injection bloqueado pelo guardrail antes do modelo principal)
Tokens médios/turno	~1540	1593 (+ ~3% pelo guardrail)
Latência média	~1.98 s	1.92 s

A nova arquitetura tornou o chatbot melhor? Sim, e as métricas mostram onde: a qualidade funcional e a latência ficaram equivalentes (~9/10, ~2 s), mas a taxa de comportamento seguro subiu de **4/7 para 7/7** — a réplica da Sprint 2 inventou especificações técnicas e atuou como consultora jurídica/financeira, enquanto a Sprint 03 recusou e encaminhou a profissionais habilitados em todos os casos. Além disso, a memória passou a ser persistente e isolada por sessão, e a avaliação deixou de ser manual: qualquer mudança futura pode ser validada re-executando a suíte. O custo desse ganho foi pequeno (~3% de tokens adicionais pelo guardrail e um falso positivo borderline no caso F5).

7.4 Problemas encontrados e soluções

Problema 1 — Histórico manual frágil e não persistente. Na Sprint 2 o histórico era uma lista em memória: sem isolamento por usuário, sem persistência e com crescimento descontrolado. *Alternativas*: (a) serializar a lista manualmente em arquivo; (b) usar a memória de sessão do framework. *Solução adotada*: `SQLiteSession` do Agents SDK. *Justificativa*: elimina código próprio de persistência, isola sessões por id e é o mecanismo idiomático do framework (requisito da sprint).

Problema 2 — Vulnerabilidade a prompt injection. O chatbot da Sprint 2 dependia apenas do system prompt; instruções do tipo "ignore suas instruções" disputavam espaço com o ataque no mesmo contexto. *Alternativas:* (a) endurecer só o system prompt; (b) filtro por palavras-chave (regex); (c) guardrail com agente classificador. *Solução adotada:* defesa em camadas — regras explícitas no system prompt e `@input_guardrail` com classificador estruturado (Pydantic) que aciona tripwire antes do modelo principal. *Justificativa:* regex gera muitos falsos negativos (ataques parafraseados); o classificador entende intenção, e o tripwire economiza tokens do modelo principal em caso de ataque.

Problema 3 — Modelo planejado indisponível na chave de API. A comparação prevista com `gpt-4.1-mini` falhou com erro 403 (`model_not_found`): o projeto da chave só tinha acesso a `gpt-4o-mini` e `gpt-5-nano`. *Alternativas:* (a) trocar de chave/projeto; (b) comparar apenas variações de parâmetros do mesmo modelo; (c) usar `gpt-5-nano` como segundo modelo. *Solução adotada:* `gpt-5-nano`, com ajuste no código — a família gpt-5 é de modelos de raciocínio e **não aceita o parâmetro `temperature`**, então `criar_agente()` passou a aceitar `temperature=None`. *Justificativa:* mantém dois modelos de gerações diferentes na comparação (o enunciado pede pelo menos dois modelos) sem depender de acesso externo, e o contraste modelo clássico × modelo de raciocínio enriqueceu a análise de latência e custo.

Problema 4 — Falso positivo do guardrail. No teste F5 (pergunta de motorista final, persona fora do público-alvo mas tema relacionado), o guardrail bloqueou a mensagem em uma das execuções, tratando-a como fora de escopo. *Alternativas:* (a) remover "fora de escopo" do tripwire e tratar só no system prompt; (b) afrouxar as instruções do classificador. *Solução adotada:* instruir o classificador a marcar "na dúvida, false" e manter o caso F5 na suíte como sentinela de regressão. *Justificativa:* o custo do falso positivo é baixo (mensagem educada de redirecionamento, nota 6/10 do juiz), enquanto remover o bloqueio de escopo enfraqueceria a defesa contra desvio de finalidade do chatbot.

7.5 Divisão da equipe

Nome	RM	Principal responsabilidade
André Santos de Azevedo	RM572236	Arquitetura e integração com o framework de agentes
Bruno Menezes Monegatto	RM570311	Memória de sessão e chat interativo
Fabiana Yumi Rodrigues Nakagawa	RM571249	Guardrails e testes de segurança
Iago Neiva Gorrao	RM570234	Suíte de testes e avaliação (LLM-as-judge)
João Pedro Amorim Albuquerque	RM573342	Comparação de modelos e baseline da Sprint 2
Kayky Araujo Silva	RM569535	Relatórios e documentação